



Задачи по информатике

Дата печати: 04.09.2026 18:32:50

Всего задач: 9

1. № 6283 (Сложный)

УСЛОВИЕ ЗАДАЧИ

№ 6283

Логическая функция F задаётся выражением

$$\neg (\neg (X \rightarrow \neg W) \wedge Z) \wedge \neg (W \rightarrow Z) \wedge (X \rightarrow \neg Z)$$

На рисунке приведён заполненный фрагмент таблицы истинности функции F, содержащий неповторяющиеся наборы аргументов

?	?	?	?	F
---	---	---	---	---

1	0	_	0	1
---	---	---	---	---

1	0	_	_	0
---	---	---	---	---

_	1	_	1	0
---	---	---	---	---

Определите, сколько существует различных способов расстановки переменных w, x, y, z подходящих для данной таблицы истинности?

ОТВЕТ

8

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun(x, y, z, w):
    return not( (x<= (not w)) and z ) and not(w <= z) and (x <= (not z))
for per in permutations('xyzw'):
    for a, b, c, d, e in product([0, 1], repeat=5):
        tab = [(1, 0, a, 0, 1), (1, 0, b, c, 0), (d, 1, e, 1, 0)]
        rez = []
        for t in tab:
            rez.append(fun(*dict(zip(per, t))))
        if len(set(tab)) == 3 and rez == [1, 0, 0]:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====

from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв xyzw
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun(x, y, z, w):
    # Определяем логическую функцию fun от 4 переменных
```

```

return not( not(x <= (not w)) and z ) and not(w <= z) and (x <= (not z))
# Вычисляем значение сложного логического выражения:
#  $\neg ( \neg ( x \rightarrow \neg w ) \wedge z ) \wedge \neg ( w \rightarrow z ) \wedge ( x \rightarrow \neg z )$ 
# =====
for per in permutations('xyzw'):
    # per - очередной вариант переставленных букв, например ('x','y','z','w') или ('x','w','z','y')
    # Всего 4! = 24 возможных перестановки
    for a, b, c, d, e in product([0, 1], repeat=5):
        # a,b,c,d,e - это неизвестные значения в таблице истинности (обозначены как ?)
        # product([0,1], repeat=5) генерирует все возможные комбинации из 5 цифр 0 или 1
        # Всего 2^5 = 32 комбинации
        tab = [(1, 0, a, 0, 1), (1, 0, b, c, 0), (d, 1, e, 1, 0)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 5 значений)
        # В каждой строке:
        # - цифры 0 и 1 - это известные значения из условия
        # - буквы a,b,c,d,e - это неизвестные значения, которые мы перебираем
        rez = []
        # Создаем пустой список для хранения результатов вычисления функции
        for t in tab:
            # t - очередная строка таблицы, например (1, 0, a, 0, 1)
            rez.append(fun(*dict(zip(per, t))))
            # Разбираем по шагам:
            # 1. zip(per, t) - соединяет буквы (xyzw) и их значения (0/1):
            #    например, при per=('x','y','z','w') и t=(1,0,a,0) получим:
            #    [('x',1), ('y',0), ('z',a), ('w',0)]
            # 2. dict(...) - превращает список кортежей в словарь: {'x':1, 'y':0, 'z':a, 'w':0}
            # 3. **dict - распаковывает словарь в аргументы функции: fun(x=1, y=0, z=a, w=0)
            # 4. Результат вычисления функции добавляем в список rez
        if len(set(tab)) == 3 and rez == [1, 0, 0]:
            # Проверяем два условия:
            # 1) len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
            #    set(tab) превращает список строк в множество, убирая дубликаты
            # 2) rez == [1,0,0] - значения функции совпадают с последним столбцом таблицы
            #    (по условию задачи в последнем столбце должны быть значения 1,0,0)
            print(*per, tab)
            # Если оба условия выполнены - нашли решение
            # Выводим: порядок перестановки букв и саму таблицу истинности для дальнейшей проверки
            # *per распаковывает перестановку, например выведет: x y z w
            break
    # Выходим из внутреннего цикла перебора комбинаций a,b,c,d,e
    # Так как решение уже найдено, дальше перебирать комбинации не нужно

```

2. № 2695 (Сложный) ✓ ★

УСЛОВИЕ ЗАДАЧИ

№ 2695 (Сложный) Сколько существует различных способов расстановки

Миша заполнял таблицу истинности функции $(w \vee y) \wedge (x \rightarrow \neg z) \wedge \neg w$, но успел заполнить лишь фрагмент из трёх её строк, даже не указав, какому столбцу таблицы соответствует каждая из переменных w, x, y, z

?	?	?	?	F
	0		0	1
1				1
1	1	0	0	1

Определите, сколько существует различных способов расстановки переменных w, x, y, z подходящих для данной таблицы истинности?4

ОТВЕТ

4

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun(x, y, z, w):
    return (w or y) and (x <= (not z)) and not w
for per in permutations('xyzw'):
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        tab = [(a, 0, b, 0, 1), (1, c, d, e, 1), (1, 1, 0, 0, 1)]
        rez = []
        for t in tab:
            rez.append(fun(*dict(zip(per, t))))
        if len(set(tab)) == 3 and rez == [1, 1, 1]:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====

from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв xyzw
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun(x, y, z, w):
    # Определяем логическую функцию fun от 4 переменных
    return (w or y) and (x <= (not z)) and not w
    # Вычисляем значение логического выражения:
    #  $(w \vee y) \wedge (x \rightarrow \neg z) \wedge \neg w$ 
    # =====
for per in permutations('xyzw'):
    # per - очередной вариант переставленных букв, например ('x','y','z','w') или ('x','w','z','y')
    # Всего 4! = 24 возможных перестановки
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        # a,b,c,d,e,f,g - это неизвестные значения в таблице истинности (обозначены как ?)
        # product([0,1], repeat=7) генерирует все возможные комбинации из 7 цифр 0 или 1
        # Всего 2^7 = 128 комбинаций
        tab = [(a, 0, b, 0, 1), (1, c, d, e, 1), (1, 1, 0, 0, 1)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 5 значений)
        # В каждой строке:
        # - цифры 0 и 1 - это известные значения из условия
```

```

# - буквы a,b,c,d,e,f,g - это неизвестные значения, которые мы перебираем
rez = []
# Создаем пустой список для хранения результатов вычисления функции
for t in tab:
    # t - очередная строка таблицы, например (a, 0, b, 0, 1)
    rez.append(fun(**dict(zip(per, t))))
    # Разбираем по шагам:
    # 1. zip(per, t) - соединяет буквы (хуzw) и их значения (0/1):
    #    например, при per=('x','y','z','w') и t=(a,0,b,0) получим:
    #    [('x',a), ('y',0), ('z',b), ('w',0)]
    # 2. dict(...) - превращает список кортежей в словарь: {'x':a, 'y':0, 'z':b, 'w':0}
    # 3. **dict - распаковывает словарь в аргументы функции: fun(x=a, y=0, z=b, w=0)
    # 4. Результат вычисления функции добавляем в список rez
if len(set(tab)) == 3 and all(fun(**dict(zip(per, c))) == c[-1] for c in tab):
    # Проверяем два условия:
    # 1) Len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
    #    set(tab) превращает список строк в множество, убирая дубликаты
    # 2) Для каждой строки c в таблице tab значение функции fun
    #    совпадает с последним элементом строки (значением F)
    print(*per, tab)
    # Если оба условия выполнены - нашли решение
    # Выводим: порядок перестановку букв и саму таблицу истинности для дальнейшей проверки
    # *per распаковывает перестановку, например выведет: x y z w
    break
    # Выходим из внутреннего цикла перебора комбинаций a,b,c,d,e,f,g
    # Так как решение уже найдено, дальше перебирать комбинации не нужно

```

3. № 332 дз (Сложный)

УСЛОВИЕ ЗАДАЧИ

№ 332 дз (Сложный) Сколько существует различных способов расстановки

Для приведенного фрагмента таблицы истинности выражения $F = (x \vee y \wedge \neg z) \wedge \neg w$ определите количество возможных последовательностей имен столбцов.

?	?	?	?	F
1	0	0	0	1
0	0	1	0	1
0	1	0	1	0

В ответе укажите только количество таких комбинаций.4

ОТВЕТ

4

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun(x, y, z, w):
    return (x or y and (not z)) and (not w)
for per in permutations('xyzw'):
    for a in product([0, 1], repeat=0):
        tab = [(1, 0, 0, 0, 1), (0, 0, 1, 0, 1), (0, 1, 0, 1, 0)]
        rez = []
        for t in tab:
            rez.append(fun(**dict(zip(per, t))))
        if len(set(tab)) == 3 and rez == [1, 1, 0]:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====

from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв xyzw
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun(x, y, z, w):
    # Определяем логическую функцию fun от 4 переменных
    return (x or y and (not z)) and (not w)
    # Вычисляем значение логического выражения:
    # (x ∨ y ∧ ¬z) ∧ ¬w
# =====
for per in permutations('xyzw'):
    # per - очередной вариант переставленных букв, например ('x', 'y', 'z', 'w') или ('x', 'w', 'z', 'y')
    # Всего 4! = 24 возможных перестановки
    for a in product([0, 1], repeat=0):
        # В этой таблице все значения известны, поэтому переменные для перебора не нужны (repeat=0)
        tab = [(1, 0, 0, 0, 1), (0, 0, 1, 0, 1), (0, 1, 0, 1, 0)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 5 значений)
        # Все значения в таблице известны
        rez = []
        # Создаем пустой список для хранения результатов вычисления функции
        for t in tab:
            # t - очередная строка таблицы, например (1, 0, 0, 0, 1)
```

```

rez.append(fun(**dict(zip(per, t))))
# Разбираем по шагам:
# 1. zip(per, t) - соединяет буквы (хуzw) и их значения (0/1):
#    например, при per=('x','y','z','w') и t=(1,0,0,0) получим:
#    [('x',1), ('y',0), ('z',0), ('w',0)]
# 2. dict(...) - превращает список кортежей в словарь: {'x':1, 'y':0, 'z':0, 'w':0}
# 3. **dict - распаковывает словарь в аргументы функции: fun(x=1, y=0, z=0, w=0)
# 4. Результат вычисления функции добавляем в список rez
if len(set(tab)) == 3 and rez == [1, 1, 0]:
    # Проверяем два условия:
    # 1) len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
    #    set(tab) превращает список строк в множество, убирая дубликаты
    # 2) rez == [1,1,0] - значения функции совпадают с последним столбцом таблицы
    #    (по условию задачи в последнем столбце должны быть значения 1,1,0)
    print(*per, tab)
    # Если оба условия выполнены - нашли решение
    # Выводим: порядок перестановки букв и саму таблицу истинности для дальнейшей проверки
    # *per распаковывает перестановку, например выведет: x y z w
    break
# Выходим из внутреннего цикла перебора комбинаций
# Так как решение уже найдено, дальше перебирать не нужно

```

4. № 6989 дз (Средний)

УСЛОВИЕ ЗАДАЧИ

№ 6989 дз (Средний) Сколько существует различных способов расстановки Логическая функция F задаётся выражением $x \rightarrow y \wedge z$. На рисунке приведён фрагмент таблицы истинности функции F , содержащий неповторяющиеся наборы аргументов.

x	y	z	F
0	1	0	0
1	1	0	0

Определите, сколько существует различных способов расстановки переменных x, y, z

ОТВЕТ

2

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun(x, y, z):
    return x <= (y and z)
for per in permutations('xyz'):
    for a in product([0, 1], repeat=0):
        tab = [(0, 1, 0, 0), (1, 1, 0, 0)]
        rez = []
        for t in tab:
            rez.append(fun(**dict(zip(per, t))))
        if len(set(tab)) == 2 and rez == [0, 0]:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====

from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв xyz
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun(x, y, z):
    # Определяем логическую функцию fun от 3 переменных
    return x <= (y and z)
    # Вычисляем значение логического выражения:
    #  $x \rightarrow y \wedge z$ 
# =====
for per in permutations('xyz'):
    # per - очередной вариант переставленных букв, например ('x','y','z') или ('x','z','y')
    # Всего 3! = 6 возможных перестановок
    for a in product([0, 1], repeat=0):
        # В этой таблице все значения известны, поэтому переменные для перебора не нужны (repeat=0)
        tab = [(0, 1, 0, 0), (1, 1, 0, 0)]
        # Создаем таблицу из 2 строк (каждая строка - кортеж из 4 значений)
        # Все значения в таблице известны
        rez = []
        # Создаем пустой список для хранения результатов вычисления функции
        for t in tab:
            # t - очередная строка таблицы, например (0, 1, 0, 0)
            rez.append(fun(**dict(zip(per, t))))
```



```

# Разбираем по шагам:
# 1. zip(per, t) - соединяет буквы (xyz) и их значения (0/1):
#    например, при per=('x','y','z') и t=(0,1,0) получим:
#    [('x',0), ('y',1), ('z',0)]
# 2. dict(...) - превращает список кортежей в словарь: {'x':0, 'y':1, 'z':0}
# 3. **dict - распаковывает словарь в аргументы функции: fun(x=0, y=1, z=0)
# 4. Результат вычисления функции добавляем в список rez
if len(set(tab)) == 2 and rez == [0, 0]:
    # Проверяем два условия:
    # 1) len(set(tab)) == 2 - все строки таблицы разные (нет повторяющихся)
    #    set(tab) превращает список строк в множество, убирая дубликаты
    # 2) rez == [0,0] - значения функции совпадают с последним столбцом таблицы
    #    (по условию задачи в последнем столбце должны быть значения 0,0)
    print(*per, tab)
    # Если оба условия выполнены - нашли решение
    # Выводим: порядок перестановки букв и саму таблицу истинности для дальнейшей проверки
    # *per распаковывает перестановку, например выведет: x y z
    break
    # Выходим из внутреннего цикла перебора комбинаций
    # Так как решение уже найдено, дальше перебирать не нужно

```

5. № 12671 дз (Средний)

УСЛОВИЕ ЗАДАЧИ

№ 12671 дз (Средний) БАЗОВЫЙ КОД – ПОРЯДОК ДЕЙСТВИЙ!!!

Логическая функция F задаётся выражением $\neg(x \equiv w \wedge \neg z) \wedge (y \equiv x \wedge \neg w)$. На рисунке приведён частично заполненный фрагмент таблицы истинности функции F , содержащий неповторяющиеся строки.

Определите, какому столбцу таблицы истинности соответствует каждая из переменных w, x, y, z .

?	?	?	?	F
		0		1
	0		0	1
0		1	0	1
Wxyz				

ОТВЕТ

Wxyz

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun(x, y, z, w):
    return not (x == (w and (not z))) and (y == (x and not w))
for per in permutations('xyzw'):
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        tab = [(a, b, 0, c, 1), (d, 0, e, 0, 1), (0, f, 1, 0, 1)]
        rez = []
        for t in tab:
            rez.append(fun(*dict(zip(per, t))))
        if len(set(tab)) == 3 and rez == [1, 1, 1]:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====
from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв xyzw
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun(x, y, z, w):
    # Определяем логическую функцию fun от 4 переменных
    return not (x == (w and (not z))) and (y == (x and not w))
    # Вычисляем значение логического выражения:
    #  $\neg(x \equiv w \wedge \neg z) \wedge (y \equiv x \wedge \neg w)$ 
# =====
for per in permutations('xyzw'):
    # per - очередной вариант переставленных букв, например ('x','y','z','w') или ('x','w','z','y')
    # Всего 4! = 24 возможных перестановки
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        # a,b,c,d,e,f,g - это неизвестные значения в таблице истинности (обозначены как ?)
        # product([0,1], repeat=7) генерирует все возможные комбинации из 7 цифр 0 или 1
        # Всего 2^7 = 128 комбинаций
        tab = [(a, b, 0, c, 1), (d, 0, e, 0, 1), (0, f, 1, 0, 1)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 5 значений)
        # В каждой строке:
        # - цифры 0 и 1 - это известные значения из условия
```

```

# - буквы a,b,c,d,e,f,g - это неизвестные значения, которые мы перебираем
# Последний столбец - значение функции F
rez = []
# Создаем пустой список для хранения результатов вычисления функции
for t in tab:
    # t - очередная строка таблицы, например (a, b, 0, c, 1)
    rez.append(fun(**dict(zip(per, t))))
    # Разбираем по шагам:
    # 1. zip(per, t) - соединяет буквы (хуzw) и их значения (0/1):
    #    например, при per=('x','y','z','w') и t=(a,b,0,c) получим:
    #    [('x',a), ('y',b), ('z',0), ('w',c)]
    # 2. dict(...) - превращает список кортежей в словарь: {'x':a, 'y':b, 'z':0, 'w':c}
    # 3. **dict - распаковывает словарь в аргументы функции: fun(x=a, y=b, z=0, w=c)
    # 4. Результат вычисления функции добавляем в список rez
if len(set(tab)) == 3 and rez == [1, 1, 1]:
    # Проверяем два условия:
    # 1) Len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
    #    set(tab) превращает список строк в множество, убирая дубликаты
    # 2) rez == [1,1,1] - значения функции совпадают с последним столбцом таблицы
    #    (по условию задачи в последнем столбце должны быть значения 1,1,1)
    print(*per, tab)
    # Если оба условия выполнены - нашли решение
    # Выводим: порядок перестановки букв и саму таблицу истинности для дальнейшей проверки
    # *per распаковывает перестановку, например выведет: x y z w
    break
# Выходим из внутреннего цикла перебора комбинаций a,b,c,d,e,f,g
# Так как решение уже найдено, дальше перебирать комбинации не нужно

```

6. № 13077 (Средний)

УСЛОВИЕ ЗАДАЧИ

□ № 13077 (Средний) 2 ФУНКЦИИ – ЗНАЧЕНИЯ не СОВПАДАЮТ

Две логические функции заданы выражениями:

$$F1 = (w \equiv x) \wedge (y \rightarrow z)$$

$$F2 = (w \rightarrow x) \rightarrow (y \equiv z)$$

Дан частично заполненный фрагмент, содержащий неповторяющиеся строки таблицы истинности обеих функций.

?	?	?	?	F1	F2
1		1	1	1	0
	1	0	0	1	
	0	0		0	0

Определите, какому столбцу таблицы истинности соответствует каждая из переменных w, x, y, z.

В ответе напишите буквы w, x, y, z в том порядке, в котором идут соответствующие им столбцы (сначала буква, соответствующая первому столбцу; затем буква, соответствующая второму столбцу, и т. д.). Буквы в ответе пишите подряд, никаких разделителей между буквами ставить не нужно. Zywx

ОТВЕТ

Zywx

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun1(x, y, z, w):
    return (w == x) and (y <= z)
def fun2(x, y, z, w):
    return (w <= x) <= (y == z)
for per in permutations('xyzw'):
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        tab = [(1, a, 1, 1, 1, 0), (b, 1, 0, 0, 1, c), (d, 0, 0, e, 0, 0)]
        rez1 = []
        rez2 = []
        for t in tab:
            rez1.append(fun1(**dict(zip(per, t))))
            rez2.append(fun2(**dict(zip(per, t))))
        if len(set(tab)) == 3 and rez1 == [1, 1, 0] and rez2 == [0, 1, 0]:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====
from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв хууу
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun1(x, y, z, w):
    # Определяем первую функцию fun1 от 4 переменных
    return (w == x) and (y <= z)
    # Вычисляем значение логического выражения:
    # (w ≡ x) ∧ (y → z)
# =====
def fun2(x, y, z, w):
```

```

# Определяем вторую функцию fun2 от 4 переменных
return (w <= x) <= (y == z)
# Вычисляем значение логического выражения:
#  $(w \rightarrow x) \rightarrow (y \equiv z)$ 
# =====
for per in permutations('xyzw'):
    # per - очередной вариант переставленных букв, например ('x','y','z','w') или ('x','w','z','y')
    # Всего 4! = 24 возможных перестановки
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        # a,b,c,d,e,f,g - это неизвестные значения в таблице истинности (обозначены как ?)
        # product([0,1], repeat=7) генерирует все возможные комбинации из 7 цифр 0 или 1
        # Всего 2^7 = 128 комбинаций
        tab = [(1, a, 1, 1, 1, 0), (b, 1, 0, 0, 1, c), (d, 0, 0, e, 0, 0)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 6 значений)
        # В каждой строке:
        # - цифры 0 и 1 - это известные значения из условия
        # - буквы a,b,c,d,e,f,g - это неизвестные значения, которые мы перебираем
        # Последние два столбца - значения F1 и F2
        rez1 = []
        rez2 = []
        # Создаем пустые списки для хранения результатов вычисления функций
        for t in tab:
            # t - очередная строка таблицы, например (1, a, 1, 1, 1, 0)
            rez1.append(fun1(**dict(zip(per, t))))
            rez2.append(fun2(**dict(zip(per, t))))
            # Разбираем по шагам:
            # 1. zip(per, t) - соединяет буквы (xyzw) и их значения (0/1):
            #    например, при per=('x','y','z','w') и t=(1,a,1,1) получим:
            #    [('x',1), ('y',a), ('z',1), ('w',1)]
            # 2. dict(...) - превращает список кортежей в словарь: {'x':1, 'y':a, 'z':1, 'w':1}
            # 3. **dict - распаковывает словарь в аргументы функции: fun1(x=1, y=a, z=1, w=1)
            # 4. Результат вычисления функции добавляем в список rez1
        if len(set(tab)) == 3 and rez1 == [1, 1, 0] and rez2 == [0, 1, 0]:
            # Проверяем условия:
            # 1) len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
            #    set(tab) превращает список строк в множество, убирая дубликаты
            # 2) rez1 == [1,1,0] - значения первой функции совпадают с предпоследним столбцом
            # 3) rez2 == [0,1,0] - значения второй функции совпадают с последним столбцом
            print(*per, tab)
            # Если условия выполнены - нашли решение
            # Выводим: порядок перестановки букв и саму таблицу истинности
            # *per распаковывает перестановку, например выведет: x y z w
            break
        # Выходим из внутреннего цикла перебора комбинаций
        # Так как решение уже найдено, дальше перебирать не нужно

```

7. № 9357 дз (Средний)

УСЛОВИЕ ЗАДАЧИ

□ № 9357 дз (Средний) 2 ФУНКЦИИ – ЗНАЧЕНИЯ СОВПАДАЮТ

Миша заполнял таблицу истинности для двух функций

$(x \rightarrow y) \vee (\neg w \equiv z)$ и $(x \rightarrow y) \equiv (w \wedge \neg z)$,

но успел заполнить лишь фрагмент из трёх различных её строк, даже не указав, какому столбцу таблицы соответствует каждая из переменных w,x,y,z.

?	?	?	?
–	–	–	0
–	–	0	0
–	0	0	0

Определите, какому столбцу таблицы истинности функции F соответствует каждая из переменных x,y,z,w, если известно, что значения функций для наборов из фрагмента равны.

Wyxz

ОТВЕТ

Wyxz

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun1(x, y, z, w):
    return (x <= y) or (not w == z)
def fun2(x, y, z, w):
    return (x <= y) == (w and not z)
for per in permutations('xyzw'):
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        tab = [(a, b, c, 0), (d, e, 0, 0), (f, 0, 0, 0)]
        rez1 = []
        rez2 = []
        for t in tab:
            rez1.append(fun1(**dict(zip(per, t))))
            rez2.append(fun2(**dict(zip(per, t))))
        if len(set(tab)) == 3 and rez1 == rez2:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====

from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв хууу
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun1(x, y, z, w):
# Определяем первую функцию fun1 от 4 переменных
    return (x <= y) or (not w == z)
    # Вычисляем значение логического выражения:
    # (x → y) ∨ (¬w ≡ z)
# =====
def fun2(x, y, z, w):
# Определяем вторую функцию fun2 от 4 переменных
    return (x <= y) == (w and not z)
```

```

# Вычисляем значение логического выражения:
#  $(x \rightarrow y) \equiv (w \wedge \neg z)$ 
# =====
for per in permutations('xyzw'):
    # per - очередной вариант переставленных букв, например ('x','y','z','w') или ('x','w','z','y')
    # Всего 4! = 24 возможных перестановки
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        # a,b,c,d,e,f,g - это неизвестные значения в таблице истинности (обозначены как ?)
        # product([0,1], repeat=7) генерирует все возможные комбинации из 7 цифр 0 или 1
        # Всего 2^7 = 128 комбинаций
        tab = [(a, b, c, 0), (d, e, 0, 0), (f, 0, 0, 0)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 4 значений)
        # В каждой строке:
        # - цифры 0 и 1 - это известные значения из условия
        # - буквы a,b,c,d,e,f,g - это неизвестные значения, которые мы перебираем
        # В таблице только столбцы переменных, значения функций не указаны
        rez1 = []
        rez2 = []
        # Создаем пустые списки для хранения результатов вычисления функций
        for t in tab:
            # t - очередная строка таблицы, например (a, b, c, 0)
            rez1.append(fun1(**dict(zip(per, t))))
            rez2.append(fun2(**dict(zip(per, t))))
            # Разбираем по шагам:
            # 1. zip(per, t) - соединяет буквы (xyzw) и их значения (0/1):
            #    например, при per=('x','y','z','w') и t=(a,b,c,0) получим:
            #    [('x',a), ('y',b), ('z',c), ('w',0)]
            # 2. dict(...) - превращает список кортежей в словарь: {'x':a, 'y':b, 'z':c, 'w':0}
            # 3. **dict - распаковывает словарь в аргументы функции: fun1(x=a, y=b, z=c, w=0)
            # 4. Результат вычисления функции добавляем в список rez
        if len(set(tab)) == 3 and rez1 == rez2:
            # Проверяем два условия:
            # 1) len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
            #    set(tab) превращает список строк в множество, убирая дубликаты
            # 2) rez1 == rez2 - значения функций для наборов из фрагмента равны
            print(*per, tab)
            # Если оба условия выполнены - нашли решение
            # Выводим: порядок перестановки букв и саму таблицу истинности для дальнейшей проверки
            # *per распаковывает перестановку, например выведет: x y z w
            break
        # Выходим из внутреннего цикла перебора комбинаций a,b,c,d,e,f,g
        # Так как решение уже найдено, дальше перебирать комбинации не нужно

```

8. № 6837 (Средний)

УСЛОВИЕ ЗАДАЧИ

□ № 6837 (Средний) ** 5 переменных – неизвестны числа в таблице

Дана логическая функция пяти логических переменных:

$$f(x_1, x_2, x_3, x_4, x_5) = (x_1 \vee \neg x_2 \vee \neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_1 \vee \neg x_2 \vee x_3 \vee x_4 \vee x_5) \wedge (x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4 \vee x_5)$$

и фрагмент таблицы истинности этой функции с неизвестными значениями некоторых ячеек.

x1	x2	x3	x4	x5	f
0	1	1	0	a	1
0	1	1	1	0	b
0	1	c	d	1	0
0	0	0	1	0	e

Укажите в ответе значения неизвестных величин a,b,c,d,e подряд без пробелов и запятых.

00101

ОТВЕТ

00101

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
def fun(x,y,z,w,q): return (x or not y or (not z) or w or (not q)) and \
    ((not x) or (not y) or z or w or q) and (x or (not y) or (not z) or (not w) or q)
fl=True
for per in permutations('xyzwq'):
    if fl==False: break
    for a, b, c, d, e, f in product([0, 1], repeat=6):
        tab = [(0,1,1,0,a,1), (0,1,1,1,0,b), (0,1,c,d,1,0), (0,0,0,1,0,e)]
        rez=[]
        for t in tab:
            rez.append(fun(**dict(zip(per, t))))
        if len(set(rez))==4 and rez==[1,b,0,e]:
            # print(*per,tab)
            print(a,b,c,d,e)
            fl=False
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв xyzwq
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun(x, y, z, w, q):
    # Определяем логическую функцию fun от 5 переменных
    return (x or not y or (not z) or w or (not q)) and \
        ((not x) or (not y) or z or w or q) and (x or (not y) or (not z) or (not w) or q)
    # Вычисляем значение сложного логического выражения:
    # (x ∨ ¬y ∨ ¬z ∨ w ∨ ¬q) ∧ (¬x ∨ ¬y ∨ z ∨ w ∨ q) ∧ (x ∨ ¬y ∨ ¬z ∨ ¬w ∨ q)
    # =====
fl = True
# Флаг для остановки перебора после нахождения решения
for per in permutations('xyzwq'):
    # per - очередной вариант переставленных букв, например ('x','y','z','w','q') или ('x','w','z','y','q')
    # Всего 5! = 120 возможных перестановок
    if fl == False: break
```



```

# Если решение уже найдено, прекращаем перебор перестановок
for a, b, c, d, e, f in product([0, 1], repeat=6):
    # a,b,c,d,e,f - это неизвестные значения в таблице истинности (обозначены как ?)
    # product([0,1], repeat=6) генерирует все возможные комбинации из 6 цифр 0 или 1
    # Всего 2^6 = 64 комбинации
    tab = [(0, 1, 1, 0, a, 1), (0, 1, 1, 1, 0, b), (0, 1, c, d, 1, 0), (0, 0, 0, 1, 0, e)]
    # Создаем таблицу из 4 строк (каждая строка - кортеж из 6 значений)
    # В каждой строке:
    # - цифры 0 и 1 - это известные значения из условия
    # - буквы a,b,c,d,e,f - это неизвестные значения, которые мы перебираем
    # Последний столбец - значение функции F
    rez = []
    # Создаем пустой список для хранения результатов вычисления функции
    for t in tab:
        # t - очередная строка таблицы, например (0, 1, 1, 0, a, 1)
        rez.append(fun(**dict(zip(per, t))))
        # Разбираем по шагам:
        # 1. zip(per, t) - соединяет буквы (хуzywq) и их значения (0/1):
        #    например, при per=('x','y','z','w','q') и t=(0,1,1,0,a) получим:
        #    [('x',0), ('y',1), ('z',1), ('w',0), ('q',a)]
        # 2. dict(...) - превращает список кортежей в словарь: {'x':0, 'y':1, 'z':1, 'w':0, 'q':a}
        # 3. **dict - распаковывает словарь в аргументы функции: fun(x=0, y=1, z=1, w=0, q=a)
        # 4. Результат вычисления функции добавляем в список rez
    if len(set(tab)) == 4 and rez == [1, b, 0, e]:
        # Проверяем два условия:
        # 1) len(set(tab)) == 4 - все строки таблицы разные (нет повторяющихся)
        #    set(tab) превращает список строк в множество, убирая дубликаты
        # 2) rez == [1, b, 0, e] - значения функции совпадают с последним столбцом таблицы
        #    В последнем столбце значения: 1 (из первой строки), b (из второй строки),
        #    0 (из третьей строки), e (из четвертой строки)
        print(a, b, c, d, e)
        # Выводим искомые значения a,b,c,d,e
        fl = False
        # Устанавливаем флаг для остановки перебора
        break
    # Выходим из внутреннего цикла перебора комбинаций a,b,c,d,e,f
    # Так как решение уже найдено, дальше перебирать комбинации не нужно

```

9. № 5424 (Сложный)

УСЛОВИЕ ЗАДАЧИ

№ 5424 (Сложный) ** БАЗОВЫЙ КОД в таблице порядок известен, а в функции нет
Логическая функция F задаётся выражением $(p_3 \rightarrow p_1) \rightarrow (p_4 \vee \neg p_2)$. На рисунке приведён частично заполненный фрагмент таблицы истинности функции F , содержащий неповторяющиеся строки. Определите, какой столбец в таблице каждой переменной в выражении.

x	y	z	w	F
0	0		1	0
0	1		1	0
1	1			0

В ответе напишите буквы x,y,z,w в том порядке, который соответствует переменным p1,p2,p3 и p4 в выражении. Буквы в ответе пишете подряд, никаких разделителей между буквами ставить не нужно.ywxz

ОТВЕТ

ywxz

КОД (БЕЗ КОММЕНТАРИЕВ)

```
from itertools import *
fun=lambda a,b,c,d: (c <= a) <= (d or not b)
print(*'xyzw')
for per in permutations('abcd'):
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        tab = [(0, 0, a, 1, 0), (0, 1, b, 1, 0), (1, 1, c, d, 0)]
        rez = []
        for t in tab:
            rez.append(fun(**dict(zip(per, t))))
        if len(set(tab)) == 3 and rez == [0, 0, 0]:
            print(*per, tab)
            break
```

КОД (С КОММЕНТАРИЯМИ)

```
# =====

from itertools import *
# Импортируем функции для работы с функциями:
# permutations - для перебора всех перестановок букв p1,p2,p3,p4
# product - для перебора всех возможных комбинаций 0 и 1
# =====
def fun(p1, p2, p3, p4):
    # Определяем логическую функцию fun от 4 переменных
    return (p3 <= p1) <= (p4 or not p2)
    # Вычисляем значение логического выражения:
    # (p3 → p1) → (p4 ∨ ¬p2)
# =====
for per in permutations(['p1', 'p2', 'p3', 'p4']):
    # per - очередной вариант переставленных переменных, например ('p1','p2','p3','p4') или ('p1','p3','p2',
    # Всего 4! = 24 возможных перестановки
    for a, b, c, d, e, f, g in product([0, 1], repeat=7):
        # a,b,c,d,e,f,g - это неизвестные значения в таблице истинности (обозначены как ?)
        # product([0,1], repeat=7) генерирует все возможные комбинации из 7 цифр 0 или 1
        # Всего 2^7 = 128 комбинаций
        tab = [(0, 0, a, 1, 0), (0, 1, b, 1, 0), (1, 1, c, d, 0)]
        # Создаем таблицу из 3 строк (каждая строка - кортеж из 5 значений)
        # В каждой строке:
```

```

# - цифры 0 и 1 - это известные значения из условия
# - буквы a,b,c,d,e,f,g - это неизвестные значения, которые мы перебираем
# Последний столбец - значение функции F
rez = []
# Создаем пустой список для хранения результатов вычисления функции
for t in tab:
    # t - очередная строка таблицы, например (0, 0, a, 1, 0)
    rez.append(fun(*dict(zip(per, t))))
    # Разбираем по шагам:
    # 1. zip(per, t) - соединяет имена переменных (p1,p2,p3,p4) и их значения (0/1):
    #    например, при per=('p1','p2','p3','p4') и t=(0,0,a,1) получим:
    #    [('p1',0), ('p2',0), ('p3',a), ('p4',1)]
    # 2. dict(...) - превращает список кортежей в словарь: {'p1':0, 'p2':0, 'p3':a, 'p4':1}
    # 3. **dict - распаковывает словарь в аргументы функции: fun(p1=0, p2=0, p3=a, p4=1)
    # 4. Результат вычисления функции добавляем в список rez
if len(set(tab)) == 3 and rez == [0, 0, 0]:
    # Проверяем два условия:
    # 1) Len(set(tab)) == 3 - все строки таблицы разные (нет повторяющихся)
    #    set(tab) превращает список строк в множество, убирая дубликаты
    # 2) rez == [0,0,0] - значения функции совпадают с последним столбцом таблицы
    #    (по условию задачи в последнем столбце должны быть значения 0,0,0)
    print(*per, tab)
    # Если оба условия выполнены - нашли решение
    # Выводим: порядок перестановки переменных и саму таблицу истинности для дальнейшей проверки
    # *per распаковывает перестановку, например выведет: p1 p2 p3 p4
    break
# Выходим из внутреннего цикла перебора комбинаций a,b,c,d,e,f,g
# Так как решение уже найдено, дальше перебирать комбинации не нужно

```